

SAIR_DA_JAULA > start
 cobra > string
 carneirinho > int
 camaleao > float
 vagalume > bool
 alcateia > array/list
 contar_carneirinho > while
 rugir("mensagem") > print
 zebra > try se_der_zembra > else
 gato > import (pq fazer gato)
 capturar > input
 bicho_solto > null
 habitat > dict / map
 break > fugir
 abater > delete ou drop
 CHEGOU_O_ZELADOR > return
 migrar > for
 fossil > const
 morto > false
 vivo > true
 se_tem_bicho > if
 Se_for_outro_bixo > if else
 nao_tem_bicho > else
 hakuna_matata > pass

Comando / Tipo	ZooLang	Equivalente C / Python	Significado / Duplo Sentido
Início do Programa	ENTRAR_NA_JAULA	int main() {	Entra na jaula para começar o treino.
Fim / Retorno	SAIR_DA_JAULA	return 0; }	Finaliza a execução e sai da jaula.
Texto	cobra	string / char*	Longa, contínua e cheia de caracteres.

Inteiro	carneirinho	int	Da piada de contar carneirinhos pra dormir.
Decimal	camaleao	float / double	Muda de valor conforme as casas decimais.
Booleano	vagalume	bool	Tipo que liga (vivo) ou desliga (morto).
Constante	fossil	const	Dado antigo, petrificado, que nunca muda.
Verdadeiro / Falso	vivo / morto	True / False	Estados booleanos do vagalume.
Lista / Array	alcateia	list / Array	Grupo organizado do mesmo tipo de dado.
Dicionário / Map	habitat	dict / Map	Um território que guarda chave/valor.
Nulo	bicho_solto	None / null	Valor sem dono, solto no sistema.
Importar	gato <lib>	import / #include	Fazer um "gato" (gambiarra) pra puxar libs.

Saída (Print)	<code>rugir("...")</code>	<code>print()</code> / <code>printf()</code>	Solta o som do monstro no terminal.
Entrada (Input)	<code>capturar(var)</code>	<code>input()</code> / <code>scanf()</code>	Captura a presa (dados do usuário).
Condicionais	<code>se_tem_bicho</code>	<code>if</code>	Testa se a condição é verdadeira.
Condicionais	<code>Se_for_outro_bixo</code>	<code>else if</code>	Testa uma condição alternativa.
Condicionais	<code>nao_tem_bicho</code>	<code>else</code>	Caminho padrão quando não há bicho.
Laço Enquanto	<code>contar_carneirinho</code>	<code>while</code>	Repete continuamente até dar o sono.
Laço Para	<code>migrar</code>	<code>for</code>	Percorre um intervalo ou lista em bando.
Interrupção	<code>fugir</code>	<code>break</code>	Foge do loop imediatamente.
Exceções	<code>zebra</code> / <code>se_der_zebra</code>	<code>try</code> / <code>catch</code>	Testa algo perigoso; trata se "der zebra".

Deletar Memória	<code>abater</code>	<code>del / free()</code>	Remove a variável na faca.
------------------------	---------------------	---------------------------	----------------------------

Manual de Sintaxe & Exemplos

1. O "Hello World" da Selva

Snippet de código

```
ENTRAR_NA_JAULA
  rugir("QUE SEANRA EH ESSA?! AQUI EH ZOOLANG!")
SAIR_DA_JAULA
```

Explicação:

Todo programa em ZooLang precisa iniciar com `ENTRAR_NA_JAULA` e encerrar com `SAIR_DA_JAULA`. O comando `rugir()` imprime o texto passado entre parênteses diretamente no terminal.

2. Variáveis, Constantes e Gambiarras (`gato`)

Snippet de código

```
gato math

ENTRAR_NA_JAULA
  fossil carneirinho LIMITE_CAGE = 100
  cobra nome_do_monstro = "Bambam"
  carneirinho idade = 35
  camaleao peso_supino = 120.5
  vagalume ta_saindo_da_cadeia = vivo
  bicho_solto suplemento_vazio = NADA

  rugir("Monstro: " + nome_do_monstro)
SAIR_DA_JAULA
```

Explicação:

- `gato math`: Usa a gíria brasileira de "fazer um gato" (uma ligação clandestina/gambiarra) para importar a biblioteca de matemática.
- `fossil`: Define uma constante (não pode ter seu valor alterado).
- `cobra`, `carneirinho`, `camaleao` e `vagalume`: Representam `string`, `int`, `float` e `bool`.
- `vivo` / `morto`: São os valores booleanos literais.
- `bicho_solto`: Inicializa uma variável como nula (`null`).

3. Entrada de Dados e Condicionais

Snippet de código

```
ENTRAR_NA_JAULA
  carneirinho carga
  rugir("Quanto voce levanta no supino?")
```

```

capturar(carga)

se_tem_bicho (carga >= 100) {
  rugir("TA SAINDO DA CADEIA O MONSTRO!")
} Se_for_outro_bixo (carga >= 60) {
  rugir("Ta mediano, da pra melhorar!")
} nao_tem_bicho {
  rugir("Carga de frango! VAI EMBORA!")
}
SAIR_DA_JAULA

```

Explicação:

- **capturar(carga)**: Lê um valor digitado no teclado pelo usuário e guarda na variável **carga**.
- **se_tem_bicho**: Estrutura condicional **if**.
- **Se_for_outro_bixo**: Estrutura condicional **else if**.
- **nao_tem_bicho**: Estrutura **else** padrão.

4. Coleções (**alcateia** e **habitat**)

Snippet de código

```

ENTRAR_NA_JAULA
alcateia exercicios = ["Supino Reto", "Agachamento 200kg", "Rosca 21"]

habitat treino_da_semana = {
  "segunda": "Peito de Aço",
  "terca": "Perna de Elefante"
}

rugir("Primeiro exercicio: " + exercicios[0])
rugir("Treino de segunda: " + treino_da_semana["segunda"])
SAIR_DA_JAULA

```

Explicação:

- **alcateia**: Funciona como um **array** ou **list** (um bando de elementos ordenados).
- **habitat**: Funciona como um **dicionário** ou **map** (um território estruturado com pares de chave e valor).

5. Laços de Repetição (**contar_carneirinho** e **migrar**) e **fugir**

Snippet de código

```

ENTRAR_NA_JAULA
carneirinho sono = 0

~ Repetição com While ~
contar_carneirinho (sono < 10) {
  se_tem_bicho (sono == 5) {

```

```

        rugir("Cansou! Foge do loop!")
        fugir
    }
    rugir("Contando carneirinho número: " + sono)
    sono = sono + 1
}

~ Repetição com For ~
migrar (carneirinho i = 1; i <= 3; i = i + 1) {
    rugir("Migrando no bando, passo: " + i)
}
SAIR_DA_JAULA

```

Explicação:

- **contar_carneirinho**: Funciona como o laço **while**. A piada vem do hábito de contar carneirinhos repetidamente até dormir.
- **migrar**: Funciona como o laço **for** clássico com inicializador, condição e incremento.
- **fugir**: Equivale ao comando **break**. Interrompe o loop antes da hora e "foge da jaula".

6. Tratamento de Erros (**zebra**) e Limpeza (**abater**)

Snippet de código

```

ENTRAR_NA_JAULA
carneirinho divisor = 0

zebra {
    carneirinho resultado = 100 / divisor
    rugir("Resultado: " + resultado)
} se_der_zebra {
    rugir("DEU ZEBRA! Nao da pra dividir por zero!")
}

abater divisor
SAIR_DA_JAULA

```

Explicação:

- **zebra { ... }**: Tenta executar um bloco de código sujeito a falhas (equivalente ao **try**).
- **se_der_zebra { ... }**: Captura a exceção caso algo dê errado/deu zebra (equivalente ao **catch / except**).
- **abater**: Remove a variável da memória (equivalente ao **del** em Python ou **free()** em C).

Exemplo Completo Final

Snippet de código

```
~ ===== ~
~ PROGRAMA PRINCIPAL EM ZOOLANG ~
~ ===== ~
gato math
```

ENTRAR_NA_JAULA

```
cobra atleta = "Bambam"
fossil carneirinho META = 100
carneirinho carga
vagalume esta_pronto = vivo

rugir("Qual a carga no supino?")
capturar(carga)

zebra {
    se_tem_bicho (carga >= META) {
        rugir(atleta + " BATEU A META DE " + META + "KG! MONSTRO!")
    } Se_for_outro_bixo (carga >= 50) {
        rugir("Ta no caminho certo, vai subir o peso!")
    } nao_tem_bicho {
        rugir("Carga de frango!")
    }
} se_der_zebra {
    rugir("DEU ZEBRA NO EXERCICIO!")
}

carneirinho serie = 1
contar_carneirinho (serie <= 3) {
    rugir("Serie " + serie + " concluida!")
    serie = serie + 1
}

abater carga
rugir("TREINO FINALIZADO! TÁ PAGO PORRA!")
```

SAIR_DA_JAULA